ACTIVIDAD 8.

INTRODUCCIÓN

Este ejercicio consiste en desarrollar un programa que gestione los productos de una tienda aplicando los concpetos de la programación orientada a objetos (POO). A lo largo de esta actividad he implementado conceptos como encapsulamiento, herencia, uso de interfaces y organización del código mediante el uso de clases.

OBJETIVO

El objetivo es permitir la gestión de productos de tipo comida y bebida. Hacer ingresos de productos, mostrar productos o calcular precios totales son algunas de las funciones que nuestro programa tiene que ser capaz de hacer. Para esto utilizaremos un menú interactivo.
Otro de los objetivos a nivel de programación es estructurar el código de forma clara y reutilizable.
PD: Este documento es navegable. *Los títulos tienen hipervinculos que direccionan hacia GitHub dentro del proyecto.*

ESTRUCTURA DEL PROYECTO

El proyecto está organizado en varias clases:

- **Producto**: Esta clase continentemente los atributos comunes (nombre, precio y cantidad). Esta clase actua como base del sistema. Ademas, incluye el método obtenerPrecioTotal(), que calcula el precio total en funición del precio y la cantidad, aplicando encapsulamiento al no requerir parámetros externos.
- **Comida:** Esta clase hereda de producto, añade atributos específicos como caducidad, tipo y tipo de cocción. También implementa la interfaz Coccion, obligando asi a definir el método tipoCoccion() dentro de la clase.
- **Bebida:** Esta clase hereda de producto. Aprovecha los métodos heredados sin necesidad de redefinirlos. Añade atributos especificos como unidad y sabor.
- **Coccion:** Esta clase es una interfaz. Obliga a todas las clases que la implementen a definir el método tipoCoccion (int tipo); dentro de ellas.
- **Tienda:** Esta clase no venia en el ejercicio. La he implementado yo porque por una cuestión de organización necesito una clase 'cerebro' que contenga los métodos que serán llamados por el método main dentro de la clase Main.
- **Main:** Esta clase será la encargada de llamar a los métodos para ejecutarlos. Contendrá un menú interactivo usando do/while, try/catch y switch/case.

FUNCIONAMIENTO DEL PROGRAMA

El programa se ejecuta a través de un menú interactivo que permite al usuario:
1- añadir productos del tipo comida
2- añadir productos del tipo bebida
3- mostrar el stock de productos almacenados

Los productos se almacenan en arrays dentro de la clase Tienda, y se gestionan mediante contadores qeu indican la posición de inserción. Exactamente de la misma forma que lo hice en la actividad 7. Usando el patron array + contador + metodos. El sistema incluye validaciones básicas para evitar errores en la entrada de datos.

No entro a desgranar los métodos uno a uno porque mantienen exactamente la misma estructura que la actividad7 entregada justo antes de este ejercicio.

CONCLUSIONES

Esta vez la batalla ha sido implementar bien la herencia y la interfaz que pedia el ejercicio. He tenido un par de choques con la herencia por duplicar métodos pero he podido solventarlo y entenderlo. Otra cosa es la construcción de los getters y los setters asi como los constructores. Haciendo ctrl + i lo hago todo… esto dificulta aprenderme la sintaxis de los metodos get y set y los constructores ya que netbeans tiene herramientas para ello. No se si debería revisar este aspecto más adelante.
Lo más importante es que este ejercicio me ha permitido aplicar de forma práctica los conceptos fundamentales de las POO, herencia, encapsulamiento y la separación de responsabilidades. Ademas de trabajar en la organizacion del código para facilitar su mantenimiento y comprensión.

Ahora seguimos con el bloque del Output.

BLOQUE DEL OUTPUT

Ahora vamos a ver el funcionamiento del programa, los errores que pueda dar y como los solventamos.

```
run:
-----MENÚ------

1. Ingresar comida
2. Ingresar bebida
3. Mostrar stock
0. Salir
1
Nombre de la comida:
Tofu
Ingresa el precio:
3.99
entrada inválida
-----MENÚ------

1. Ingresar comida
2. Ingresar bebida
3. Mostrar stock
0. Salir
1
Nombre de la comida:
Tofu
Ingresa el precio:
3,99
Ingresa la cantidad:
3
Ingresa la caducidad:
3/9/2036
Ingresa el tipo de comida
preparado de soja
Ingresa el tipo de cocción 1.sarten/2.horno/3.Microondas
1
```

Como se puede ver al teclear mal el double salta la excepcion y nos devuelve al principio de la acción. Una vez bien introducida nos deja meter los datos. Ponemos los de la comida.

```
1
-----MENÚ------

1. Ingresar comida
2. Ingresar bebida
3. Mostrar stock
0. Salir
2
Ingresa nombre
Bebida de avena
Ingresa precio
2,15
Ingresa cantidad
4
Ingresa las unidades
4
Ingresa el sabor
avena
```

Introducimos los datos de la bebida usando la opción 2 y los podemos meter sin problema.

```
avena
-----MENÚ------

1. Ingresar comida
2. Ingresar bebida
3. Mostrar stock
0. Salir
3
--- STOCK ---
-Comidas-

Comida: 1Tofu
Precio: 3.99
Precio total: 11.97
Tipo de cocción: 0


Bebida: 1Bebida de avena
Precio: 2.15
```

Y por último opción 3 mostrar stock. *

```
-----MENÚ------

1. Ingresar comida
2. Ingresar bebida
3. Mostrar stock
0. Salir
0
Adiós
BUILD SUCCESSFUL (total time: 6 minutes 58 seconds)
```

Y por último salir del programa. Se cierra perfectamente.

CONCLUSIÓN PERSONAL

Este trabajo era casi calcado al anterior. La diferencia ha sido la herencia y la implementación de una interfaz. Es muy fácil duplicar getters y setters y que entren en conflicto. Es algo a vigilar.
El tiempo de trabajo ha sido menor al anterior. Unas 6 horas entre código y documentacion. La conclusión es la misma de las anteriores: **hay que estudiar mucho.**

POSTCREDITOS

* me di cuenta que el Output en este caso no es el que pide el ejercicio asi que vamos arreglarlo.

La 'incidencia' está dentro de la clase Tienda que es la que contiene las funciones del programa. Dentro de mostrarStock() en el segundo for reza lo siguiente:

```java
    for (int i = 0; i < contadorBebidas; i++) {
        Bebida b = bebida[i];
        System.out.println();
        System.out.println("Bebida: " + (i + 1) + b.getNombre());
        System.out.println("Precio: " + b.getPrecio());
        System.out.println();
    }
```

```java
for (int i = 0; i < contadorBebidas; i++) {
    Bebida b = bebida[i];
    System.out.println();
    System.out.println("Bebida: " + (i + 1) + b.getNombre());
    System.out.println("Precio: " + b.getPrecio());
    System.out.println("Precio total: " + b.obtenerPrecioTotal());
    System.out.println();
}
```

```
run:
-----MENÚ------

1. Ingresar comida
2. Ingresar bebida
3. Mostrar stock
0. Salir
2
Ingresa nombre
Bebida de avena
Ingresa precio
3,99
Ingresa cantidad
3
Ingresa las unidades
3
Ingresa el sabor
avena
-----MENÚ------

1. Ingresar comida
2. Ingresar bebida
3. Mostrar stock
0. Salir
3
--- STOCK ---
-Comidas-

Bebida: 1Bebida de avena
Precio: 3.99
Precio total: 11.97
```

Como se puede ver faltan rótulos. Así que vamos a agregarlos.

Una vez arreglado el código ya funciona perfectamente.

Nota adicional: siguiendo las instrucciones del ejercicio use el atributo cantidad y unidades pero no le veo el uso. De todos modos ahí queda.

BIBLIOGRAFÍA CONSULTADA:
Documentación: clases Linkia, imprimibles Linkia.
Herramientas de aprendizaje: NoteBook LM, Chat GPT.